Document Number: P0290R0
Date: 2016-02-19
Author: Anthony Williams
Just Software Solutions Ltd

P0290R0: `apply()` for `synchronized_value<T>`

Introduction

This paper is a followup to N4033, based on feedback from the Rapperswil 2014 meeting.

The basic idea is that `synchronized_value<T>` stores a value of type`T` and a mutex.The `apply()` function then provides a means of accessing the stored value with the mutex locked, automatically unlocking the mutex afterwards.

The `apply()` function is variadic, so you can operate on a set of `synchronized_value<T>` objects. All the mutexes are locked prior to invoking the supplied callable object, and then they are all released afterwards.

This provides an easy way for developers to ensure that all accesses to a given object are done with the relevant mutex locked, whilst also allowing for operations that require locks on multiple objects.

In order to avoid simple mistakes when using the `synchronized_value<T>` objects, there are no public member functions or operations other than construction.

Examples

1: Simple accesses

Simple accesses can be done with simple lambdas:

```
synchronized_value<std::string> s;

std::string read_value(){
    return apply([](auto& x){return x;},s);
}

void set_value(std::string const& new_val){
    apply([&](auto& x){x=new_val;},s);
}
```

2: More complex processing

More complex processing can be done with a more complex lambda, or a separate function or callable object:

```
synchronized_value<std::queue<message_type>> queue;

void process_message(){
    std::optional<message_type> local_message;
    apply([&](std::queue<message_type>& q){
        if(!q.empty()){
            local_message.emplace(std::move(q.front()));
            q.pop_front();
        }
    },queue);
    if(local_message)
        do_processing(local_message.value());
}
```

3: Multi-value processing

The variadic nature of `apply()` means that writing code that accesses multiple `synchronized_value<T>` objects is straightforward. It uses the same mechanism as `std::lock()` to ensure that the requisite mutexes are locked without deadlock.

The ubiquitous example of transferring money between accounts can then be simply written as a follows:

```
void transfer_money(
    synchronized_value<account>& from_,
```

```
    synchronized_value<account>& to_,
    money_value amount){
    apply([=](auto& from,auto& to){
        from.withdraw(amount);
        to.deposit(amount);
    },from_,to_);
}
```

Proposed wording

Add a new section to chapter 30 as follows.

30.x Synchronized Values

This section describes a class template to associate a mutex (30.4) with a value in order to facilitate the construction of race-free programs.

Header <synchronized_value> synopsis

```
namespace std {
    template<typename T>
    class synchronized_value;

    template<typename F,typename ... ValueTypes>
    decltype(std::declval()(std::declval()...)) apply(
        F&& f,synchronized_value<ValueTypes>&... values);
}
```

30.x.1 Class template `synchronized_value`

```
namespace std
{
    template<typename T>
    class synchronized_value
    {
    public:
        synchronized_value(synchronized_value const&) = delete;
        synchronized_value& operator=(synchronized_value const&) = delete;

        template<typename ... Args>
        synchronized_value(Args&& ... args);
        ~synchronized_value();
    };
}
```

An object of type `synchronized_value<T>` wraps an object of type `T` along with a mutex to ensure that only one thread can access the wrapped object at a time. The wrapped object can be accessed by passing a callable object or function to `apply`. All such accesses are done with the internal mutex locked.

`template<typename ... Args>`
`synchronized_value(Args&& ... args);`

Requires:
: `T` is constructible from `args`

Effects:
: Constructs a `std::synchronized_value` instance containing an object constructed with `T(std::forward<Args>(args)...)`. If no arguments are supplied then the wrapped object is default-constructed.

Throws:
: Any exceptions thrown by the construction of the wrapped object.

`~synchronized_value();`

Effects:
: Destroys `*this` and the contained object of type `T`.

30.x.2 `apply` function

```
template<typename F,typename ... ValueTypes>
decltype(std::declval()(std::declval()...)) apply(
    F&& f,synchronized_value<ValueTypes>&... values);
```

Effects:

Locks the internal mutex for each V_i in `values` using a sequence of calls that does not result in deadlock (see [thread.lock.algorithm]). Then invokes `func(data...)`, where each $\texttt{data}_i$ is the wrapped object of type $\texttt{ValueType}_i$ stored in V_i, and returns the result to the caller. When the invocation of `func` returns or exits via exception, then the internal mutexes in each V_i are unlocked prior to control leaving `apply`.

Returns:

The return value of the call to `func`.

Throws:

`std::system_error` if any of the locks could not be acquired. Any exceptions thrown by the invocation of `func(data...)`. Note: the internal mutexes are unlocked after the call, even if `func(data...)` exits with an exception.

Synchronization:

Multiple threads may call `apply()` concurrently without external synchronization. If multiple threads call `apply()` concurrently passing the same instance(s) of `synchronized_value` then the behaviour is as-if they each made their call in some unspecified order. The completion of the full expression associated with one access synchronizes-with a subsequent access to the same object.